CS315 · ALGORITHMS ANALYSIS & DESIGN

Lecture 5 Greedy Algorithms

Midterm summary · CLRS Chapter 16

Muharib

What's inside

01 The idea

02 Elements of the greedy strategy

03 Activity-selection

04 Knapsack

05 Huffman codes

01 The idea

Greedy takes the best option available right now, never reverses a decision, and does not check whether that choice leads to the overall optimum.

> A greedy algorithm selects the best option available at the moment. It never reverses an earlier decision, even if the choice is wrong. It works in a **top-down** approach.

Applications: activity selection, knapsack and Huffman codes.

02 Elements of the greedy strategy

Property	Meaning
Greedy-choice property	A globally optimal solution is reached by a locally optimal (greedy) choice, without considering subproblem results.
Optimal substructure	An optimal solution contains optimal solutions to subproblems (shared with DP).

	Greedy	Dynamic programming
Decision	One choice now, never revisited	Tries all choices and stores results
Direction	Top-down	Usually bottom-up
Optimum	Not always	Always (when both properties hold)

03 Activity-selection

Activities all need the same resource (one hall). Each has a start `s` and a finish `f`. We want the largest set of non-overlapping activities.

Two activities are **compatible** if their intervals [si, fi) and [sj, fj) do not overlap.
Greedy choice: sort by finish time; pick the activity that finishes first, remove the ones that overlap it, repeat.

Slide example

a	1	2	3	4	5	6	7	8	9	10	11
s	1	3	0	5	3	5	6	8	8	2	12
f	4	5	6	7	9	9	10	11	12	14	16

```
take a1 (finishes at 4)
a2, a3 start before 4 → rejected
a4 starts at 5 ≥ 4 → take a4 (finishes 7)
a5, a6, a7 start before 7 → rejected
a8 starts at 8 ≥ 7 → take a8 (finishes 11)
a9, a10 start before 11 → rejected
a11 starts at 12 ≥ 11 → take a11
```

```
Answer: {a1, a4, a8, a11}  →  4 activities
```

Other sets of the same size exist, such as `{a2, a4, a9, a11}`.

```
GREEDY-ACTIVITY-SELECTOR(s, f)
1  n = s.length
2  A = {a1}
3  k = 1
4  for m = 2 to n
5      if s[m] ≥ f[k]
6          A = A ∪ {am}
7          k = m
8  return A
```

Running time

Both the iterative and recursive versions are $\Theta(n)$ if the activities are already sorted by f, since each activity is examined once. Unsorted input adds the $O(n \log n)$ sort.

"The recursive activity-selection algorithm takes O(n log n)" → **Other value**: Θ(n)

04 Knapsack

A knapsack holds at most weight W. Each item has a value v and a weight w. Maximize the value.

	Fractional knapsack	0-1 knapsack
Items	Parts of an item may be taken	Whole item or nothing (0 or 1)
Solved by	**Greedy** (optimal)	**DP** (greedy may fail)
Time	$O(n \lg n)$ due to sorting	

The greedy method: compute $d = \frac{v}{w}$ (value per unit weight), sort from largest, and take in order until full.

Slide example: W = 50

Item	w	v	$d = \frac{v}{w}$
1	10	60	6
2	20	100	5
3	30	120	4

0-1, greedy:	take 1 and 2 →	60 + 100 = 160 (20 lbs left empty)
0-1, optimal:	take 2 and 3 →	100 + 120 = 220
other option:	take 1 and 3 →	60 + 120 = 180
Fractional:	$1 + 2 + \frac{20}{30}$ of 3 →	60 + 100 + 80 = 240

So greedy fails for 0-1 (empty space lowers the value). DP is needed there: each item is tried as included and excluded, and the subproblems overlap.

Asked in the midterm

462: saffron, cardamom, cinnamon and pepper in a 5 kg bag → 5100. 461 fractional knapsack → 60. Both are solved in the solution files.

05 Huffman codes

Goal: compress text. ASCII gives every character 8 bits; Huffman gives frequent characters

short codes and rare ones longer codes.

The method

1. Write each character's count and sort ascending.
2. Take the two smallest and join them under a new node.
3. Put the new node back and repeat until one node (the root) remains.
4. Left = 0, right = 1; the code is the path from the root to the letter.

Slide example: BCCABBDDAECCBBAEDDCC (20 letters)

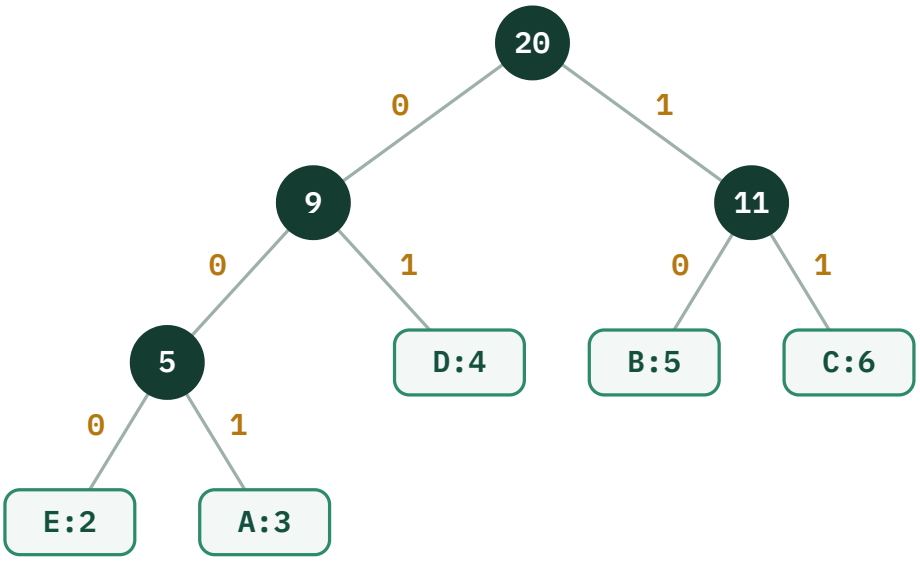
```
Counts:  E = 2,  A = 3,  D = 4,  B = 5,  C = 6
```

```
E + A = 5
```

```
5 + D = 9
```

```
B + C = 11
```

```
9 + 11 = 20  (root)
```



Letter	Count	Code	Bits
A	3	001	3 × 3 = 9
B	5	10	5 × 2 = 10
C	6	11	6 × 2 = 12
D	4	01	4 × 2 = 8
E	2	000	2 × 3 = 6

```
Huffman:  9 + 10 + 12 + 8 + 6 = 45 bits
```

```
ASCII:    20 letters × 8 = 160 bits
```

Memorize

Time complexity of Huffman coding: **O(n log n)**, n = number of unique characters.
The code length depends on the frequency of occurrence of characters. → **True**
No code is a prefix of another code (prefix code).

```
HUFFMAN(C)
1  n = |C|
2  Q = C                          // min-priority queue by frequency
3  for i = 1 to n − 1
4      z = new node
5      z.left  = x = EXTRACT-MIN(Q)
6      z.right = y = EXTRACT-MIN(Q)
7      z.freq = x.freq + y.freq
8      INSERT(Q, z)
9  return EXTRACT-MIN(Q)          // the root
```